The Limits of Read-Restricted Dependency Summaries

Vidhun Vijayakumar Suja

Independent Researcher, India
vsvidhun06@gmail.com

Abstract. Dependency analyses show which registers an instruction's result depends on and then chain these summaries. Cousot (SAS 2019) showed that such chaining is sound but can become less accurate because of cancellation, which his reduced product with linear equalities can recover. For straight-line 32-bit register programs, we determine what an instruction-local summary must retain so that composition can decide load-to-store dependence exactly. Whether dependence holds is a simple yes-or-no question for each register pair, but composition requires much more information. Against arbitrary contexts, an exact code must determine each instruction's denotation; against affine contexts, it must determine its coefficient vector modulo 2^{32}. Thus, no summary containing only an instruction's target and a subset of its read registers (taint, occurrence, or local non-constancy) can be exact under any combining scheme, regardless of the oracle used locally. Conversely, Cousot's composition is exact when the relevant steps do not combine two load-derived values, reuse a derived value non-injectively, or allow a side input to annihilate it. Symbolic composition, using composed labels, is exact for the events of a thread. All numbered theorems are formally proved in Isabelle/HOL without additional axioms or oracles. A prototype and an independent reference confirm the predicted separation on constructed examples, but the evidence from real code is negative. Across 2,250 litmus tests and eight upstream concurrency code bases and test suites, we found no composed cancellation in a store's data dependencies. The 52 cancellations found in control or address dependencies do not change the examined verdicts, by structural argument. Under $\mathsf{acyclic}(\mathsf{sdep} \cup \mathsf{rf})$, composed labels admit 14 constructed load-buffering programs that the closure forbids, 13 of which clang compiles to load-independent stores. Because the criterion is per-thread, it still forbids outcomes that Promising allows according to our derivation.

Keywords: Semantic dependency · Weak memory models · Thin-air reads · Isabelle/HOL · SMT

1 Introduction

A store *depends* on a load if changing the loaded value can change the stored value. Analyses rarely decide this on the whole program. They summarise each instruction by the registers its result depends on and chain the summaries along register

Table 1. The worked example. Both threads read y and store s to x. Each local summary lists the registers whose value can change that instruction's result, other registers free. The composed label replaces registers by their definitions back to the load's input symbol L. "LB" is the two-thread program in which each thread is a copy of the column, cross-wired (x and y swapped in the second copy), asking for $r_0{=}r_1{=}1$.

	T_+ (cancellation)	$T_\times$ (doubling)
code	$r \leftarrow \texttt{load}(y)$; $t := r{+}1$; $s := t{-}r$; $\texttt{store}(x, s)$	$r \leftarrow \texttt{load}(y)$; $t := r{+}r$; $s := t{-}r$; $\texttt{store}(x, s)$
local summaries	$t : \{r\}$, $s : \{t, r\}$	$t : \{r\}$, $s : \{t, r\}$ (identical)
chained closure	store depends on load	store depends on load
composed store label	$(L{+}1){-}L$, constantly 1	$(L{+}L){-}L$, equal to L
true dependence	**none**	**yes**

definitions. Taint tracking unions operand labels [37]. Syntactic dependency calculi for weak memory record that a register *occurs* in an expression [3,1]. Cousot's abstract semantic dependency [11] keeps, per assignment, the variables whose change can change the result, and composes these relations in program order. He proves the composition sound and shows by an example ($y := x$; $y := y{-}x$) that it is incomplete. This paper asks how far that incompleteness goes:

> *How much information must an instruction-local summary retain for its composition to determine load-to-store dependencies exactly?*

Table 1 shows the information lost during composition. In T_+ and $T_\times$, every instruction is non-constant in each register it reads. Consequently, summaries based on non-constancy or syntactic occurrence are identical for both threads, as is anything computed from those summaries. However, T_+ always stores 1, whereas $T_\times$ stores the loaded value. This distinction matters for memory models that prevent thin-air behaviour by forbidding cycles in $\mathsf{rf} \cup \mathsf{sdep}$, including the C++ proposal P0422 [27], modular relaxed dependencies (MRD) [29], and pomsets with predicate transformers (PwT) [21]. In the load-buffering (LB) program built from $T_\times$, the outcome requires a dependency cycle, while the corresponding program built from T_+ has no such cycle. These models compute sdep using values, preconditions, or expressions. Our lower bounds explain why this additional information is needed and why read-set analyses, including taint tracking, occurrence-based calculi, and Boolean closures, cannot be exact in general.

What this paper adds. Cousot showed that one composition rule is incomplete. We go further by identifying what every exact composition rule must retain. Dependence is a one-bit attribute for each register pair, yet composition makes it fully discriminating. Against arbitrary contexts, an exact per-instruction code must determine the instruction's denotation. Against affine contexts, it must determine its coefficient vector modulo 2^{32}. A read-set summary stores only one bit for each read register, so it cannot be exact under arbitrary composition. Even

access to an exact oracle for individual instructions cannot solve this problem. The limitation lies in how the information is represented, not in the local decision. Conversely, Cousot's composition is exact under conditions that exclude three sources of imprecision: combining two values derived from the load, reusing a non-injectively derived value, or allowing a side input to annihilate a dependency. Symbolic composition, which preserves the instruction's denotation through composed labels, is exact for the events of a thread.

Contributions. Our contributions are classified as machine-checked theorems [T], measurements of our prototype [P], or applications [A].

(1) [T] *Lower bounds* (Thms. 1–4): No read-set summary is exact under any combining function. A per-instruction code is exact if and only if it determines the instruction's denotation. For blocks, even the exact dependence relation is not sufficient as an exact summary. These bounds concern the information that must be retained, not computation time.
(2) [T] *When less information suffices* (Thms. 5–6): The closure is exact under conditions that avoid the three sources of imprecision described above. Over $\mathbb{Z}/2^{32}$, it is exact on affine code, where the coefficient vector contains exactly the information an exact code must retain. Affine analysis modulo 2^w is classical [28]; our contributions are the dependence interpretation and the capacity bounds.
(3) [T] *Composed labels* (Thm. 7): On a single thread, a store depends on a load if and only if the store's composed label is non-constant in that load's symbol. PwT [21] and Symbolic MRD [34] already use symbolic methods. Our contribution is the mechanised event-level statement, which, together with Thm. 3, shows why preserving the denotation is necessary for an exact instruction-local code.
(4) [P] *Evidence* (§4): We provide conformance checks using constructed programs, alongside a negative result from real code. Across 2,250 litmus tests and eight upstream concurrency code bases and test suites, we found no composed cancellation in a store's data dependencies. We also identified 52 cancellations in control or address dependencies, which do not change the examined verdicts according to our structural argument.
(5) [A] *Load buffering* (§5): Under $\mathsf{acyclic}(\mathsf{sdep} \cup \mathsf{rf})$, composed labels admit constructed load-buffering programs that the closure forbids, including programs that clang compiles to load-independent stores. However, because the analysis is per-thread, it retains dependencies that cancel only through memory or value-range constraints. Consequently, the criterion is not invariant under store forwarding.

Scope. These theorems apply to data dependencies in single-threaded, straight-line programs operating on 32-bit values. Conditional branches, loops, read-modify-writes (RMWs), address dependencies, and the multi-thread consistency criterion are outside their scope (§7).

2 Straight-Line Programs, Summaries and Labels

All definitions are the Isabelle ones in mathematical notation (Table 4, App. D).

Expressions and programs. Expressions are $e ::= n \mid r \mid {\sim}e \mid -e \mid e \circ e \mid e \bowtie e$ with $\circ \in \{+, -, *, \oplus, \&, |\}$ over 32-bit words with wrap-around, and signed relations $\bowtie \in \{=, \neq, <, \leq, >, \geq\}$ evaluating to 0 or 1. Register names are strings, an unbounded type. $[\![e]\!]_s$ evaluates e in an environment s, and $\mathit{regs}(e)$ is the set of registers that occur in e. An expression is *non-constant in* r, $\mathit{nc}(e, r)$, if $\exists s, v_1, v_2.\ [\![e]\!]_{s[r \mapsto v_1]} \neq [\![e]\!]_{s[r \mapsto v_2]}$; this is Cousot's per-assignment dependency [11, rule (10)] at 32 bits. A *program* $P = [(x_1, e_1), \ldots, (x_n, e_n)]$ is a list of assignments run in order, with final environment $\mathit{run}\, P\, s$. *Program-level dependence* asks whether the final b moves with the initial a:

$$\mathit{prog_dep}(P, a, b) \iff \exists s, v_1, v_2.\ \mathit{run}\, P\, (s[a \mapsto v_1])\, b \neq \mathit{run}\, P\, (s[a \mapsto v_2])\, b.$$

Instruction-local summaries. A marking predicate $\beta : \mathit{Expr} \times \mathit{Reg} \to \mathbb{B}$ induces the summary $\alpha_\beta(P) = [(x_i, \{r \mid \beta(e_i, r)\})]_i$. This summary records each instruction's target and the selected registers, rather than retaining the expressions themselves. With $\beta = \mathit{nc}$, we obtain the Boolean summary α. The class

$$\mathcal{A}_{\text{read}} = \{\beta \mid \forall e, r.\ \beta(e, r) \Rightarrow r \in \mathit{regs}(e)\}$$

includes predicates that mark only registers occurring in an expression: non-constancy, syntactic occurrence, taint tracking, non-zero affine coefficients, and our prototype's classifiers. The restriction is syntactic: even $0 * z$ mentions z. More generally, a code $c : \mathit{Expr} \to C$ induces $\alpha_c(P) = [(x_i, c(e_i))]_i$. A code is exact if there exists a function f such that $\mathit{prog_dep}(P, a, b) = f(\alpha_c(P), a, b)$ for all programs P and registers a, b. The function f need not be computable: exactness measures the information represented in a summary, not the computational resources required to use it.

Chained closures. Cousot's construction [11, rules (10), (16.b)] composes one dependency relation per assignment in program order. After $x_i := e_i$, every register r for which $\mathit{nc}(e_i, r)$ holds reaches x_i, while every other variable maps to itself. For a final register b, this ordered composition gives the version-aware closure $\mathit{vdep}(P, b)$, which identifies the initial registers that b can be traced back to (Isabelle: `crel_iff_vdep`):

$$\mathit{vdep}([], b) = \{b\}, \qquad \mathit{vdep}((z, e) :: Q, b) = \bigcup_{w \in \mathit{vdep}(Q, b)} \begin{cases} \{r \mid \mathit{nc}(e, r)\}, & w = z, \\ \{w\}, & w \neq z. \end{cases}$$

An assignment invalidates the earlier value of its target. In contrast, the flow-insensitive site relation $\mathit{sites}(P) = \{(r, x_i) \mid \mathit{nc}(e_i, r)\}$, when closed reflexively and transitively, ignores assignment order. For example, in $y := x;\ y := w$, with distinct registers, $(x, y) \in \mathit{sites}(P)^*$, even though the ordered composition removes the dependency on x. This is not Cousot's construction; we include it only for comparison.

Threads, events and composed labels. A thread is an ordered sequence of instructions: `Load` r, where a register receives an external value; `Asgn` x e, which assigns an expression to a register; and `Store` r, which writes the register's current value. Instruction indices identify individual events. For a load event i preceding a store event j, $esdep(T, i, j)$ holds if there exist initial register values and values for the other loads, held fixed, such that changing the value received by load i changes the value written by store j. The composed label $\ell_T(j)$ is obtained through symbolic execution. Every initial register r is represented by a symbol I_r. A load event i introduces a fresh symbol L_i and binds it to the destination register. If the same register is loaded again, a new symbol is introduced. An assignment $x := e$ binds x to the expression e, replacing every register with its current symbolic binding. The composed label $\ell_T(j)$ is the resulting binding of the register written by store j. In Table 1, for T_+, the store's composed label is $(L_0 + 1) - L_0$.

Definition 1 (Supported fragment $\mathcal{S}$). *A thread consisting of an ordered sequence of* `Load`*,* `Asgn`*, and* `Store` *instructions, using the expression grammar defined above and operating on 32-bit values. In esdep, the initial register values and the values received by load instructions are quantified over. Theorem 7(d) considers the corresponding relation with the initial registers fixed. The fragment contains no branches, loops, read-modify-write operations (RMWs), or memory semantics.*

3 What a Summary Must Retain

Every numbered theorem in this section is a machine-checked result [T]; the one hand argument is labelled separately. Each theorem statement corresponds to its Isabelle/HOL formalisation, up to notation. The theories introduce no additional axioms to HOL, use no `sorry`, and have no axiomatised theory among their ancestors. The last two properties were also checked mechanically (Table 4).

Theorem 1 (Boolean summaries are too coarse). *Let $x \neq t$ and $x \neq y$, $P_+ = [(t, x{+}1), (y, t{-}x)]$ and $P_\times = [(t, x{+}x), (y, t{-}x)]$. Then $\alpha(P_+) = \alpha(P_\times)$, $\neg prog_dep(P_+, x, y)$ and $prog_dep(P_\times, x, y)$. Hence no function f satisfies $prog_dep(P, x, y) = f(\alpha(P), x, y)$ for all P, x, y.*

P_+ is a variant of Cousot's cancellation example (Ex. 7). The program $P_\times$ adds a second case with the same summary but a real dependency. Together, these programs turn the imprecision of one composition rule into a non-definability result: the summary cannot determine the correct dependence relation. Changing the marking predicate does not solve this problem as long as it remains read-restricted.

Theorem 2 (Read-restricted summaries are insufficient). *For every $\beta \in \mathcal{A}_{\text{read}}$, no function f satisfies $prog_dep(P, a, b) = f(\alpha_\beta(P), a, b)$ for all P, a, b.*

Proof idea. At a site that reads only x, a predicate in $\mathcal{A}_{\text{read}}$ can produce only $\emptyset$ or $\{x\}$: one bit of information. Consider the first instructions $e_A = 0$, $e_B = x$, and $e_C = x \,\&\, 1$, with continuations $g_1 = t$ and $g_2 = t \,\&\, 2$. The expression e_A reads no registers and therefore marks $\emptyset$. If either e_B or e_C also marks $\emptyset$, its summary collides with that of e_A, while the continuation g_1 distinguishes their dependencies. Otherwise, both e_B and e_C mark $\{x\}$, and the continuation g_2 distinguishes them: $(x \,\&\, 1) \,\&\, 2 = 0$, whereas $x \,\&\, 2$ varies. Thus, the read-restricted summary cannot distinguish all cases needed to determine dependence exactly (App. A). □

Theorem 2 is also a corollary of Theorem 3(b). At a site that reads only x, the code $e \mapsto \{r \mid \beta(e, r)\}$ takes at most two values, which is insufficient to determine the expression's denotation. Isabelle re-derives the result in this way (`A_read_insufficient_via_capacity`). We retain the direct proof because it explains the limitation using three explicit programs. The theorem concerns the representation α_β: an instruction target together with a subset of its read registers. A summary that attaches a 32-bit coefficient to each register is not of this form and is exact on affine code (Thm. 6). What information, then, is sufficient? Call expressions e_1 and e_2 *contextually equivalent* when $\mathit{prog_dep}(Q[e_1], a, b) = \mathit{prog_dep}(Q[e_2], a, b)$ for every choice of a, b and every single-hole context $Q[\cdot] = \mathit{pre} +\!\!+ [(z, \cdot)] +\!\!+ \mathit{post}$.

Theorem 3 (Capacity of instruction-local codes). *(a) Expressions e_1, e_2 are contextually equivalent if and only if $[\![e_1]\!] = [\![e_2]\!]$. (b) A code c is exact if and only if $c(e_1) = c(e_2)$ implies $[\![e_1]\!] = [\![e_2]\!]$. In particular, the denotation itself is exact. (c) Every exact code is injective on $\{a{\cdot}x + b \mid a, b \in \{0,1\}^{32}\}$, so it takes at least 2^{64} distinct values on this family. (d) With unboundedly many register names, some marking predicate outside $\mathcal{A}_{\text{read}}$ is exact.*

The contexts that distinguish e_1 and e_2 in part (a) use two fresh registers. Proofs of this theorem and the following results appear in App. A. Part (b) has the form of the complete-abstraction condition [17]. What is specific to dependence is that even one-bit observations force an exact code to retain the full denotation. The sufficiency direction follows the idea used by substitution-based transformers: PwT's assignment rule maps a precondition ψ to $\psi[e/r]$ [21]. Part (d) shows why the read restriction in Theorem 2 cannot simply be removed. Its proof encodes the expression into an unread register name, which requires unboundedly many available names.

The same question arises for blocks, as in procedure summaries. Call blocks P_1 and P_2 *interchangeable* if $\mathit{prog_dep}(\mathit{pre} +\!\!+ P_1 +\!\!+ \mathit{post}, a, b) = \mathit{prog_dep}(\mathit{pre} +\!\!+ P_2 +\!\!+ \mathit{post}, a, b)$ for all $\mathit{pre}, \mathit{post}, a, b$. A block summary σ is *exact* if this dependence is a function of $(\sigma(P), \mathit{pre}, \mathit{post}, a, b)$.

Theorem 4 (Block summaries). *(a) Blocks P_1, P_2 are interchangeable if and only if $\mathit{run}\, P_1 = \mathit{run}\, P_2$. (b) Every exact block summary refines the denotation, and the denotation itself is an exact block summary. (c) The exact dependence relation $\{(a, b) \mid \mathit{prog_dep}(P, a, b)\}$ is not an exact block summary: $[(t, x)]$ and $[(t, x \,\&\, 1)]$ have the same dependence relation but different denotations.*

Consequently, dependence relations such as those used in IFDS-style taint summaries [33] are not exact block summaries in this semantic sense. IFDS remains exact for its own syntactic analysis problem.

Where chaining is exact. For assignment-only programs, chaining can lose precision, but it never loses a genuine dependency. Within a restricted fragment, it loses no precision at all.

Theorem 5 (The version-aware closure). *Call P* single-read injective *(SRI) if every step (z, e) has some register r such that $regs(e) \subseteq \{r\}$ and e is injective in r. (a) For all P, x, y, $prog_dep(P, x, y) \Rightarrow x \in vdep(P, y)$, and $x \in vdep(P, y) \Rightarrow (x, y) \in sites(P)^*$. (b) If P is SRI, then $prog_dep(P, x, y) \iff x \in vdep(P, y)$ for all x, y. (c) Injectivity in each read register separately is insufficient. For $r \neq q$, let $P = [(r, r{+}q), (r, r{-}q)]$. Every step is injective in r and in q, yet $q \in vdep(P, r)$ while $\neg prog_dep(P, q, r)$. (d) The relation $sites^*$ is not exact even on SRI programs. For distinct x, w, y, the program $[(y, x), (y, w)]$ is SRI and satisfies $(x, y) \in sites^*$, but has no program-level dependence from x to y. (e) Mark registers using a forward pass starting from x, which is initially marked* injective. *A step that reads no marked register unmarks its target. If a step has exactly one marked operand r, marked injective, its target is marked injective when e is injective in r for every value of the other registers. Otherwise, it is marked* non-constant *when e is non-constant in r for every such value. All other cases are marked* unknown. *If y does not finish with the unknown mark, then $prog_dep(P, x, y) \iff x \in vdep(P, y)$. SRI programs never produce an unknown mark. The restrictions cannot be dropped: vdep is inexact on the program in part (c), on the masking program below, and on $[(u, 0), (y, x * u)]$. These cases demonstrate the effects of combining marked operands, non-injective dependence, and annihilation by a side input.*

In the masking program $[(t, x \,\&\, 1), (y, t \,\&\, 2)]$, both steps read only one register. Although $x \in vdep(P, y)$, the final result is always 0. The intermediate value t is merely non-constant, so y ends as unknown. Conversely, $[(y, x{+}w)]$ is not SRI, but the forward analysis marks y injective, and *vdep* is exact on this program. Both results are mechanised. These examples show why inexactness can arise from correlation, range restriction, or annihilation.

Chaining over a ring. The closure *vdep* computes in the Boolean semiring: $r \in vdep((z, e) :: Q, b)$ if and only if $\big(z \in vdep(Q, b) \wedge nc(e, r)\big) \vee \big(r \neq z \wedge r \in vdep(Q, b)\big)$. Cancellation requires additive inverses, which an idempotent semiring does not provide. Call an expression e *affine* if it uses only literals, registers, unary negation, bitwise complement, addition, subtraction, and multiplication by a literal. Let $coef(e, r) \in \{0, 1\}^{32}$ denote its coefficient of r, using $\sim w = -w - 1$. The *weighted closure* replaces Boolean marks with coefficients and replaces $\wedge, \vee$ with multiplication and addition modulo 2^{32}:

$$w([\,], b, r) = [r{=}b], \quad w((z, e) :: Q, b, r) = w(Q, b, z) \cdot coef(e, r) + [r{\neq}z] \cdot w(Q, b, r).$$

Theorem 6 (The weighted closure). *Let every step of P be affine. (a) For all r, b, v, s, $\mathit{run}\, P\,(s[r{\mapsto}v])\, b \;=\; w(P,b,r)\cdot v \;+\; \mathit{run}\, P\,(s[r{\mapsto}0])\, b$. Hence $\mathit{prog_dep}(P,x,b) \iff w(P,b,x)\neq 0$, and $w(P,b,x)\neq 0 \Rightarrow x\in \mathit{vdep}(P,b)$. The weighted closure uses only instruction targets and coefficient vectors. (b) For affine expressions e_1, e_2, if $\mathit{coef}(e_1)=\mathit{coef}(e_2)$, then they are interchangeable in every affine context. If $\mathit{coef}(e_1,x)\neq \mathit{coef}(e_2,x)$, the affine context $[(t,\cdot),(y,t-\mathit{coef}(e_1,x)*x)]$ with distinct x,t,y separates them. (c) A code is exact over affine programs if and only if it determines the coefficient vector of affine expressions. On the family $\{a{\cdot}x+b\}$, every such code takes at least 2^{32} distinct values, and coef takes exactly 2^{32}. By contrast, Theorem 3(c) requires 2^{64} values over arbitrary programs.*

For P_+, the weight is $1\cdot 1-1=0$. For $P_\times$, it is $1\cdot 2-1=1$. *Two context languages.* Within the family $\{a{\cdot}x+b\}$, the expressions $2x$ and $2x{+}1$ have the same coefficients. By part (b), no affine context can distinguish them through dependence. A comparison can distinguish them. For fresh registers t, u, consider the context $[(t,\cdot),(u,(t\neq 2x){*}(u{+}1))]$ from Theorem 3(a). It leaves u constantly 0 after $2x$, but produces $u{+}1$ after $2x{+}1$. An affine continuation uses t only through a term of the form $c\cdot t$. The offset b therefore changes the result by a constant without changing the dependence verdict. This is precisely the information an exact summary for affine contexts may discard. Thus, 2^{32} classes suffice for affine contexts, whereas 2^{64} are required against arbitrary contexts on this family. These numbers count the distinct code values an exact summary must recognise. They are not direct memory costs and do not bound an implementation that retains the expression itself, as substitution-based approaches do. Beyond the affine fragment, Theorem 3 shows that an exact instruction-local code cannot retain less information than the denotation. Moreover, deciding *prog_dep* is NP-complete even for one assignment (a hand argument in App. A). Consequently, no polynomial-time exact method covers the general problem unless P = NP.

Lifting to events: composed labels are exact.

Theorem 7 (Composed labels on one thread). *Let T be a thread in $\mathcal{S}$, i a load, and j a later store. (a) $\mathit{esdep}(T,i,j)$ holds if and only if $\ell_T(j)$ is non-constant in L_i. (b) Embedding P as* `Load` a; P; `Store` b *gives esdep* $\iff$ $\mathit{prog_dep}(P,a,b)$. *(c) For every $\beta\in\mathcal{A}_{\mathrm{read}}$, no function of the instruction summary (instruction kinds and order, load and store operands, assignment targets, and β-marked sets) computes esdep on all legal load–store pairs. (d) Let esdep_s denote esdep with the initial registers fixed to s, and let $\ell^s_T(j)$ be the composed label with each initial register r bound to $s(r)$. Then $\mathit{esdep}_s(T,i,j)$ holds if and only if $\ell^s_T(j)$ is non-constant in L_i. Furthermore, $\mathit{esdep}(T,i,j)$ holds if and only if $\mathit{esdep}_s(T,i,j)$ holds for some s.*

Part (a) follows because symbolic execution over distinct, injective input symbols agrees with concrete execution. Parts (b) and (c) extend Theorems 1–2 to events. The two relations in part (d) are not identical. Consider `Load` r; $t := r{*}k$; `Store` t. Here *esdep* holds, but esdep_s fails when $s(k)=0$. This distinction is mechanised.

Deciding part (a) requires one satisfiability query in quantifier-free bit-vector logic (QF_BV) per load–store pair, using self-composition [6].

4 Composed Labels in the Prototype

Everything in this section is classified as [P]: it evaluates our implementation rather than the underlying mathematics. The Isabelle development does not establish the correctness of the Java code.

Three dependency modes. The prototype processes litmus tests into events and records syntactic data, address, and control dependencies. It classifies each data edge from a load p to a store c as real or fake using three different modes. **(A)** The shipped folding classifier uses ten cancellation idioms and flat additive normalisation for each instruction. The front end then propagates these per-instruction decisions along register definitions. Code inspection shows that this implements the version-aware closure *vdep* over load events, using a marking predicate from $\mathcal{A}_{\text{read}}$. **(B)** The second mode uses the same closure, but replaces the folding classifier with an exact QF_BV oracle. The oracle uses the SMT solver Z3 [13] to compare two copies of each expression while leaving the other registers free. It determines dependence for individual instructions before the closure combines their results. **(C)** The third mode uses composed labels. The front end constructs each store's value expression over fresh symbols representing individual loads. Mode C then queries the same oracle used in Mode B to determine whether this composed expression is non-constant in the symbol associated with load p, while leaving all other load symbols free. Mode C supports threads containing data dependencies only. Files containing address or control edges, read-modify-write operations (RMWs), page-table tokens, or constructs outside the oracle's grammar or encoder are rejected, with the reason recorded (App. D). **(L)** The weighted closure from Thm. 6 is implemented in the reference tool, not in the prototype. It chains coefficient vectors forward, whereas the theorem's function w chains backward over the same per-step matrices. The equivalence of these implementations has not been mechanised. Mode L abstains when a store's value passes through a non-affine instruction.

Independent checks. A separate Python reference implementation has its own reader, parser, and evaluator, although both implementations use the same SMT solver. A "varies" result requires a concrete witness, while a "constant" result is established by a Z3 proof. For the 359 edges that both tools can parse, Z3 proves that their composed expressions are equivalent. Mode C records 684 verdicts: 202 "varies" results, each accompanied by a 32-bit witness, and 482 "constant" results, each supported by a Z3 proof. A separate solver-free validator proves 462 of the constant results and only tests the remaining 20 (App. D). Mode L is compared only against the reference implementation.

Table 2. Conformance on the constructed population: edges classified as *fake* by each mode, according to the generator's classification. A and B use Boolean closure with folding and an exact oracle, respectively; C uses composed labels. All three operate in the prototype. L is the weighted closure in the reference tool, showing edges it determines to be fake; it abstains on non-affine expressions. The final row covers loads that are overwritten or reloaded.

classification and form	pairs	A	B	L	C
real edge (all forms)	171	0	0	0	0
fake edge, single instruction	66	30	66	26	66
fake edge, chain of instructions	72	0	0	**26**	**72**
fake edge, check cases	5	4	4	1	5
no syntactic edge, check cases	3	–	–	–	–

Population. We use a fixed population of 256 single-thread C litmus files generated from templates. These templates include cancellation (for example, $t := r+K$, followed by $\pm K$ steps and then $t - r$), masking, and injective pipelines. The population also includes 14 check cases covering copies, reloads, and overwritten registers. All but two cancellation and masking templates appear in two forms: as a chain containing up to nine assignments, and as a single assignment (App. D). Because the population is constructed, its counts should not be interpreted as frequencies in real programs.

Results. Table 2 illustrates two different forms of precision loss. *Local imprecision:* the folding classifier in Mode A misses 36 of the 66 single-instruction fake dependencies. The exact local oracle in Mode B misses none. *Composition imprecision:* when the same algebra is spread across multiple assignments, even Mode B cannot recover the lost precision. It retains all 72 chain-form fake dependencies. Composed labels in Mode C remove all 72 of these fake dependencies, along with the check case $k := 0;\ s := r * k$, where the local oracle leaves k free. No mode incorrectly reports a real dependency pair as fake. Columns B and C are conformance checks rather than independent performance measurements. Thms. 1 and 5 predict the behaviour of Mode B, while Thm. 7(a) predicts Mode C when the front end is correct. Mode L decides 131 of the 314 pairs containing an edge, with results matching the reference implementation. It abstains on the remaining 183, including every $\oplus$ and masking template and the case $k := 0;\ s := r * k$. The 26 chain-form fake dependencies removed by Mode L involve additive cancellations and copies, representing the affine part of the precision loss. The reference agrees with Mode C on all 314 pairs: 143 are constant and 171 vary. None of these pairs is width-sensitive. The tools also agree on the diagnostic inputs.

Litmus tests. We analysed 2,380 herd7 and Dat3M litmus files (§5). Mode C supports 2,250 of these files, including 2,210 byte-distinct inputs. Among the unsupported files, 128 contain address edges and two have no events. The supported files contain 326 data edges across 245 files. Four are real dependencies, while 322 are fake dependencies, all having the single-instruction form $(r \oplus r) + 1$.

As a result, Modes A, B, and C agree on every edge and verdict in this corpus. The corpus therefore cannot test the cases where Modes B and C differ. A search protocol fixed before screening found no load reaching a store through two or more assignments for which Mode B retains the dependency but Mode C removes it. This negative result covers McKenney's LKMM collection, including 4,505 files new to our investigation, of which 1,898 were supported, and the 1,767 InstCombine test files in LLVM 22.1. In four InstCombine test functions selected after the search, Mode B retains a cancelled argument's dependency while Mode C removes it. InstCombine also removes that argument. However, the difference does not change a weak-memory verdict (App. D).

Concurrency code bases. We also examined real concurrency code using a protocol established before the search, with nine recorded amendments. A separate LLVM-IR screening tool (App. D) evaluated eight upstream sources: Concurrency Kit, libvsync, liburcu, mimalloc, the GenMC tests, the Dat3M benchmarks, and two Linux 7.3 user-space harnesses. The screen examined 7,406 load-to-sink sites, with helper functions inlined. Among 501 data dependencies reaching atomic or volatile stores, none was identified as a composed cancellation, and one remained undecided. The Boolean closure nevertheless retained 52 composed cancellations in other dependency categories: 50 control dependencies and two address dependencies. None of these cancellations correlates two load-derived values in the same way as T_+. Instead, they arise from maxima, remainders, masks, bit tests, or dead paths. Clang 20.1.2 removed every cancelled instance among the six sites on which it was tested. A structural argument, rather than model checking, indicates that none of these cases changes a weak-memory verdict. Therefore, composition loss is observable at the level of dependency edges, but our investigation found no instance in which it changes a weak-memory verdict.

5 Application: Load Buffering on a Single Execution

This section is [A]: it examines executions using the theorems and an encoder that has not been formally verified.

Criterion. A *candidate execution* of a branch-free program contains events, a per-thread program order po, a reads-from relation rf, a per-location coherence order co, and syntactic dependencies $\mathsf{dep} \subseteq \mathsf{po}$ (data, address, and control). For a data edge $(p, c) \in \mathsf{dep}$ from a load to a store, $(p, c) \in \mathsf{sdep}$ if and only if the composed label of c is non-constant in p's symbol. Address edges are classified by the instruction-local classifier, while control edges are maintained syntactically. Both lie outside the scope of the theorems, and Mode C rejects files containing them. For load–store data pairs of a thread in $\mathcal{S}$, sdep is $esdep_s$, where s represents the declared initial register values (zero if undeclared), by Thm. 7(d). This assumes that the front end's composed expression denotes $\ell^s_T(j)$. That assumption has not been formally established. Instead, we test whether it represents the same 32-bit function as the expression constructed by an independent reference

implementation (§4). Memory locations are erased in this analysis. The relation $esdep_s$ varies one load while holding the other loads' inputs fixed but otherwise unrestricted. It therefore disregards how coherence correlates the values observed by loads within one execution. As a result, it can over-approximate dependence and does affect one verdict in Table 3. A candidate satisfies SDEP consistency if it satisfies per-location coherence, $\mathsf{acyclic}(\mathsf{po}_{\mathrm{loc}} \cup \mathsf{rf} \cup \mathsf{co} \cup \mathsf{fr})$, RMW atomicity, and $\mathsf{acyclic}(\mathsf{sdep} \cup \mathsf{rf})$. The last axiom is not introduced by our work. It has the form discussed for C++ in P0422 [27] and is used by MRD-C11 [29] and PwT-C11 [21]. These models differ in how sdep is computed. Our criterion quantifies each load over all 32-bit values. It is therefore coarser than PwT, MRD, and Promising 2.0 on examples involving restricted value ranges. For Batty et al.'s witnesses, see [7] and App. B.

Three ways to be forbidden. The prototype encodes each test using 32-bit bit-vector values and integer rank variables for each acyclicity axiom. It then asks Z3 whether some choice of rf and co, with the outcome fixed, satisfies the model. A FORBIDDEN verdict can arise for three different reasons, which we distinguish throughout the evaluation. **(W) Well-formedness.** Well-formedness alone is unsatisfiable. Its constraints include same-location rf sources with matching values, the requested outcome, and acyclic same-location $\mathsf{po} \cup \mathsf{co}$. The outcome is either unreachable by value or violates same-location write order. **(B) Coherence base.** The coherence model without sdep already prohibits the outcome. **(D) Dependency axiom.** Only $\mathsf{acyclic}(\mathsf{sdep} \cup \mathsf{rf})$ forbids the outcome. Only category (D) directly concerns sdep.

The matched programs. Table 3 displays the separation that the criterion is intended to demonstrate. `LB-copy` and `LB-fake-xor-cycle` have the same events, po, rf, co, and dep in the candidate execution that realises the requested outcome. They differ only in what their stores compute. The coherence-only model admits both. Therefore, the FORBIDDEN verdict for `LB-copy` under SDEP arises from the dependency axiom alone (D). The $T_{\times}$ load-buffering program is prohibited by a genuine dependency cycle. For T_{+}-LB, Modes A and B retain both dependency edges because each instruction is non-constant in the register it reads. Consequently, only the dependency axiom forbids the requested outcome (D). Under composed labels, each store's label is constantly 1. The relation sdep is therefore empty, and the outcome is admitted, just as in its single-instruction counterpart `LBfd`, our name for that program. The program `LB-real` stores $r{+}1$, so no candidate execution can realise the requested outcome (W). It therefore does not witness the effect of the dependency criterion.

Compiled code (constructed programs). We constructed 14 programs of the T_{+}-LB form whose stores cancel the load over two to five instructions, together with three dependent control programs. They are represented as LKMM litmus tests and as C11 relaxed atomics (App. D). Modes A and B prohibit all 14 cancellation programs through the dependency axiom (D) alone, while Mode C and the independent reference admit them. Clang 20.1.2, using optimisation levels `-O1`

Table 3. Load-buffering programs and their executed model checks. The requested outcome is $r_0{=}r_1{=}1$, with locations initially set to zero. "Reach." indicates whether well-formedness alone is satisfiable. Coh denotes the RC11 coherence axiom $\mathsf{irreflexive}(\mathsf{hb};\mathsf{eco}^?)$, which, over relaxed accesses, reduces to plain coherence and admits LB. The artifact refers to this model as RA, although release/acquire would forbid LB. RC11 combines plain coherence with $\mathsf{acyclic}(\mathsf{po}\cup\mathsf{rf})$. The SDEP modes use folding (A), the exact local oracle (B), and composed labels (C). The final columns report the data edges retained in sdep (out of two, or four in the final row), the verdict, and the reason for each forbidden outcome. (D) indicates that the same model with $\mathsf{sdep}=\emptyset$ admits the outcome; (W) indicates a well-formedness failure; — indicates admission. $[z]$ denotes a write followed by a reload of location z.

					SDEP, modes A/B/C		
program	store writes	reach.	Coh	RC11	|sdep|	verdict	cause
`LB-copy`	r	yes	A	F	2/2/2	F/F/F	D/D/D
`LB-chain-double` ($T_\times$)	$t{:=}r{+}r;\ t{-}r$	yes	A	F	2/2/2	F/F/F	D/D/D
`LB-fake-xor-cycle`	$r\oplus r\oplus 1$	yes	A	F	0/0/0	A/A/A	—
`LBfd`, single instr.	$r{+}1{-}r$	yes	A	F	0/0/0	A/A/A	—
`LB-chain-cancel` (T_+)	$t{:=}r{+}1;\ t{-}r$	yes	A	F	2/2/**0**	F/F/**A**	D/D/—
`LB-real`	$r{+}1$	**no**	F	F	2/2/2	F/F/F	W/W/W
`LB-mem-cancel`	$[z]{:=}r{+}1;\ [z]{-}r$	yes	A	F	4/4/**4**	F/F/**F**	D/D/D

through `-O3` for AArch64, x86-64, RISC-V, and Arm32, compiles 13 of these programs to constant stores independent of their loads. It retains the loaded value in the 14th program, $((r{\cdot}r{-}r)\,\&\,1)+1$, and in all three dependent controls. A compiled AArch64 thread therefore contains a load followed by an independent store. The Arm memory model allows their load-buffering outcome, which would require a dependency-ordered-before edge to be forbidden [1]. We did not run these programs on physical hardware. The closure-based criterion would therefore classify this compilation as unsound. In contrast, every outcome forbidden by composed labels among the 17 programs is also forbidden for the corresponding compiled code under the Arm memory model.

Exact per thread, not per execution. The program T_+ demonstrates cancellation within registers. In contrast, `LB-mem-cancel` cancels a dependency through memory: $r{\leftarrow}y;\ z{:=}r{+}1;\ u{\leftarrow}z;\ x{:=}u{-}r$, with the other thread copying x to y. Coherence forces $u=r{+}1$. The store therefore writes 1. However, the composed label $L_u{-}L_r$ still varies with L_r when L_u is held fixed. Mode C consequently retains the dependency $y\to x$ and forbids the outcome (D). According to our derivation, which has not been checked by running Promising, Promising admits this outcome. When interpreted as C11 relaxed atomics, replacing the reload with the previously stored value is sound in the monotone model forbidding $hb\cup\mathsf{rf}$ cycles [39, Thm. 10] and, by the argument in App. B, in the 2011 model. After that replacement, Mode C admits the outcome. The criterion is therefore not invariant under this store-forwarding transformation. Clang retains the relaxed reload

but forwards a plain z. PwT does not validate this transformation either [21, p. 54:28].

Rings and litmus corpus. We placed each of 1,828 single-register expressions e, satisfying $e(0) = 1$, at every store in two- or three-thread load-buffering rings (App. B). SDEP admits the 534 constant expressions and forbids, through the dependency axiom (D), the 110 non-constant expressions for which $e(1) = 1$. These outcomes are not thin-air behaviours. According to our hand derivation, Promising admits them [22,25]. The folding classifier (Mode A) also forbids 385 of the 534 constant cases. Among 2,380 analysed herd7 [3] and Dat3M [15] litmus files, SDEP admits 2,345 and forbids 33. None of the forbidden outcomes is excluded by the dependency axiom alone. Modes A and B agree on every file, while Mode C agrees on the 2,250 files it supports. These comparisons are regression checks, not independent evidence supporting the dependency criterion.

6 Related Work

Dependence and information flow. Cartwright and Felleisen [9] and Podgurski and Clarke [31] distinguish semantic dependencies from syntactic ones. Cousot [11] provides the closest related work. His per-assignment dependency (rule 10) corresponds to our *nc*, while his ordered composition (rules 10 and 16.b) corresponds to our *vdep*. Consequently, the soundness part of Thm. 5(a) is his theorem applied to straight-line programs over 32-bit words. His Example 7 illustrates the precision loss described in Thm. 1. His reduced product with linear equalities [23] can recover that precision, although it operates over a field rather than modulo 2^{32}. Our work adds constraints on the information that every exact summary must retain (Thms. 2–4 and 6(b)–(c)). We also identify conditions under which Cousot's original composition is already exact (Thm. 5(e)). Karr, Müller-Olm and Seidl [28] study affine relations, including analyses that are sound and complete modulo 2^w. By our derivation, their analysis identifies zero coefficients in affine cancellations. Their work, together with that of Elder et al. [14], contains the mathematical semantics used in Thm. 6(a). Our contribution is the interpretation of these results in terms of dependence, the capacity bounds, and their mechanisation. Further related analyses are discussed in App. B. Taint tracking sometimes handles specialised expressions such as $a \oplus a$ [35], while other approaches track information flow at the bit level [41]. However, Thm. 2 shows that no read-set rule can eliminate such precision losses in general. The impossibility result depends on interpreted operators. For free schemas, static slicing is minimal [12]. Mastroeni and Zanardini [26] refine syntactic dependence through semantic reasoning, while abstract non-interference [16] parameterises dependence-related observations by the chosen observers. Our non-constancy query uses self-composition [6], and deciding this property is computationally hard in general [42].

Relaxed memory. Boehm and Demsky [8] rule out load-to-store reordering outright, while RC11 [24] forbids all po∪rf cycles. Pomsets with Preconditions [19] attach preconditions to events. In PwT [21], these preconditions are composed

through sequencing, with reads introducing fresh registers through an implication-based construction. For register-only code, PwT therefore retains the information carried by composed labels. Its independence check can also use local values, making it more precise than our non-constancy criterion. According to our derivation, PwT admits the T_{+}-LB outcome. Symbolic MRD [34] is the closest executed work. It analyses each thread using symbolic values and flattens per-write justifications into a relation used in $\mathsf{acyclic}(dp \cup \mathsf{rf})$. It eliminates dependencies through elaboration rules rather than an explicit exactness condition. Its verdict on T_{+}-LB depends on expression simplification described in supplementary material that we could not obtain. Wright et al. [40] formalise reasoning about MRD dependencies in Isabelle/HOL, taking the dependency relation as given. The current C++ proposal [36] notes that determining semantic dependency may require changing several variables simultaneously. By contrast, *prog_dep* and *esdep* perturb one variable at a time. MRD [29], Promising [22,25], Weakestmo [10], and the event-structure models proposed by Jeffrey and Riely [20] and Pichon-Pharabod and Sewell [30] handle branches and value-range constraints that are outside the scope of our analysis. The CAT models used by herd7 [3,1], IMM [32], and LKMM [2] instead order events using syntactic dependencies.

7 Limitations and Conclusion

Theorems. The theorems apply to single-threaded programs without conditional branches or loops, focusing on data dependencies over 32-bit values. The capacity bound is mechanised for one affine family, while the complete single-register count and the NP-completeness result are established through hand arguments rather than mechanised proofs. The non-definability results apply only to read-restricted set summaries. The condition established in Thm. 5(e) is sufficient for exactness, but it is not necessary. Non-constancy is evaluated by quantifying over unrestricted load values. Consequently, the analysis may retain dependencies that value-range restrictions or coherence constraints could eliminate, as discussed in §5.

Implementation and evaluation. The front end has been tested against an independent reference implementation, but its correspondence with ℓ^{s}_{T} has not been formally verified. Mode C determines data dependencies only, and Mode L's forward chaining has not been proved equivalent to the weighted closure w. The multi-thread consistency criterion, the encoder's adequacy, and the well-formedness lemma are supported by hand arguments (App. B). Comparisons with Promising and PwT are also derived analytically rather than established through tool runs. The experimental populations are constructed. The code-base screening and litmus-test searches use purposive samples rather than prevalence estimates. Some planned sources remain unscreened, and certain sites are undecided. The analysis treats calls and other memory operations as opaque, while the absence of verdict changes is established through structural arguments rather than model checking. Only one compiler was tested. Consequently, the negative findings apply to the programs examined and should not be interpreted as evidence that

such behaviour is absent from real-world programs generally. The compiled-code examples were constructed specifically for this study. Their reported outcomes are verdicts under the Arm memory model, not results obtained from running physical Arm hardware. GCC was not available for comparison, and the artifact's main driver was executed only on Windows 11.

Conclusion. Composition makes the one-bit dependency question fully discriminating. An exact instruction-local code must determine the instruction's denotation against arbitrary contexts and its coefficient vector against affine contexts. Therefore, no summary containing only an instruction's target and a subset of its read registers can compose into exact load-to-store dependence. For block summaries, nothing coarser than the denotation is sufficient in general. Chaining itself is not the fundamental obstacle. It remains exact under conditions that avoid correlation, non-injective reuse, and annihilation, and the weighted closure is exact for all affine code over 32-bit words. An exact analysis could therefore use Boolean summaries wherever the classification in Thm. 5(e) does not produce an *unknown* result, while composing expressions only in the remaining cases. This hybrid approach has not been implemented. In the code bases we examined, composition loss appeared in control and address dependencies, but not in the data dependencies of stores. According to our structural arguments, none of these cases changes a weak-memory verdict. The practical significance of this precision loss therefore remains undemonstrated. Finally, composed labels operate per thread and do not account for every constraint imposed by an entire execution. They can consequently forbid outcomes that more value-aware semantics can justify.

Use of Generative AI

Generative AI tools (Claude) were used during the preparation of this work to assist with manuscript editing, literature exploration, boilerplate scripting, and drafting auxiliary Isabelle/HOL proof tactics. All formal specifications and proofs were mechanically verified using Isabelle's proof-checking kernel, and experimental outputs were validated through automated build and test pipelines. The authors reviewed all generated content and take full responsibility for the scientific claims, formalizations, reported results, and final presentation of this manuscript.

Data Availability

The Isabelle development, prototype, reference, inputs and drivers, and the code-base screen's protocol, tools and outputs, are archived with SHA-256 manifests; the main driver re-executes the experiments and asserts their numbers (App. D). The archives are not yet publicly deposited. They are available from the author on request and will be deposited with a persistent identifier for the final version.

References

1. Alglave, J., Deacon, W., Grisenthwaite, R., Hacquard, A., Maranget, L.: Armed cats: Formal concurrency modelling at Arm. ACM Transactions on Programming Languages and Systems **43**(2), 8:1–8:54 (2021). https://doi.org/10.1145/3458926
2. Alglave, J., Maranget, L., McKenney, P.E., Parri, A., Stern, A.: Frightening small children and disconcerting grown-ups: Concurrency in the Linux kernel. In: ASPLOS 2018. pp. 405–418. ACM (2018). https://doi.org/10.1145/3173162.3177156
3. Alglave, J., Maranget, L., Tautschnig, M.: Herding cats: Modelling, simulation, testing, and data mining for weak memory. ACM Transactions on Programming Languages and Systems **36**(2) (2014). https://doi.org/10.1145/2627752
4. Amtoft, T., Banerjee, A.: Information flow analysis in logical form. In: Static Analysis (SAS 2004). pp. 100–115. LNCS, Springer (2004). https://doi.org/10.1007/978-3-540-27864-1_10
5. Assaf, M., Naumann, D.A., Signoles, J., Totel, É., Tronel, F.: Hypercollecting semantics and its application to static analysis of information flow. In: Proceedings of the 44th ACM SIGPLAN Symposium on Principles of Programming Languages (POPL 2017). pp. 874–887. ACM (2017). https://doi.org/10.1145/3009837.3009889
6. Barthe, G., D'Argenio, P.R., Rezk, T.: Secure information flow by self-composition. In: 17th IEEE Computer Security Foundations Workshop (CSFW 2004). pp. 100–114. IEEE Computer Society (2004). https://doi.org/10.1109/CSFW.2004.1310735
7. Batty, M., Memarian, K., Nienhuis, K., Pichon-Pharabod, J., Sewell, P.: The problem of programming language concurrency semantics. In: Programming Languages and Systems (ESOP 2015). Lecture Notes in Computer Science, vol. 9032, pp. 283–307. Springer, Berlin, Heidelberg (2015). https://doi.org/10.1007/978-3-662-46669-8_12
8. Boehm, H.J., Demsky, B.: Outlawing ghosts: Avoiding out-of-thin-air results. In: Proceedings of the Workshop on Memory Systems Performance and Correctness (MSPC 2014). pp. 1–6. ACM, New York, NY, USA (2014). https://doi.org/10.1145/2618128.2618134
9. Cartwright, R., Felleisen, M.: The semantics of program dependence. In: Proceedings of the ACM SIGPLAN 1989 Conference on Programming Language Design and Implementation (PLDI 1989). pp. 13–27. ACM (1989). https://doi.org/10.1145/73141.74820
10. Chakraborty, S., Vafeiadis, V.: Grounding thin-air reads with event structures. Proc. ACM Program. Lang. **3**(POPL) (2019). https://doi.org/10.1145/3290383
11. Cousot, P.: Abstract semantic dependency. In: Static Analysis (SAS 2019). LNCS, vol. 11822, pp. 389–410. Springer (2019). https://doi.org/10.1007/978-3-030-32304-2_19
12. Danicic, S., Fox, C., Harman, M., Hierons, R., Howroyd, J., Laurence, M.R.: Static program slicing algorithms are minimal for free liberal program schemas. The Computer Journal **48**(6), 737–748 (2005). https://doi.org/10.1093/comjnl/bxh121
13. de Moura, L., Bjørner, N.: Z3: An efficient SMT solver. In: Tools and Algorithms for the Construction and Analysis of Systems (TACAS 2008). Lecture Notes in Computer Science, vol. 4963, pp. 337–340. Springer (2008). https://doi.org/10.1007/978-3-540-78800-3_24
14. Elder, M., Lim, J., Sharma, T., Andersen, T., Reps, T.: Abstract domains of affine relations. ACM Trans. Program. Lang. Syst. **36**(4), 1–73 (2014). https://doi.org/10.1145/2651361

15. Gavrilenko, N., Ponce-de León, H., Furbach, F., Heljanko, K., Meyer, R.: BMC for weak memory models: Relation analysis for compact SMT encodings. In: Computer Aided Verification (CAV 2019). Lecture Notes in Computer Science, vol. 11561, pp. 355–365. Springer, Berlin, Heidelberg (2019). https://doi.org/10.1007/978-3-030-25540-4_19
16. Giacobazzi, R., Mastroeni, I.: Abstract non-interference: Parameterizing non-interference by abstract interpretation. In: Proceedings of the 31st ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL 2004). pp. 186–197. ACM, New York, NY, USA (2004). https://doi.org/10.1145/964001.964017
17. Giacobazzi, R., Ranzato, F., Scozzari, F.: Making abstract interpretations complete. Journal of the ACM **47**(2), 361–416 (2000). https://doi.org/10.1145/333979.333989
18. Hunt, S., Sands, D.: On flow-sensitive security types. In: Conference Record of the 33rd ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL 2006). pp. 79–90. ACM, New York, NY, USA (2006). https://doi.org/10.1145/1111037.1111045
19. Jagadeesan, R., Jeffrey, A., Riely, J.: Pomsets with preconditions: A simple model of relaxed memory. Proc. ACM Program. Lang. **4**(OOPSLA), 1–30 (2020). https://doi.org/10.1145/3428262
20. Jeffrey, A., Riely, J.: On thin air reads: Towards an event structures model of relaxed memory. In: Proceedings of the 31st Annual ACM/IEEE Symposium on Logic in Computer Science (LICS 2016). pp. 759–767. ACM, New York, NY, USA (2016). https://doi.org/10.1145/2933575.2934536
21. Jeffrey, A., Riely, J., Batty, M., Cooksey, S., Kaysin, I., Podkopaev, A.: The leaky semicolon: Compositional semantic dependencies for relaxed-memory concurrency. Proc. ACM Program. Lang. **6**(POPL), 54:1–54:30 (2022). https://doi.org/10.1145/3498716
22. Kang, J., Hur, C.K., Lahav, O., Vafeiadis, V., Dreyer, D.: A promising semantics for relaxed-memory concurrency. In: Proceedings of the 44th ACM SIGPLAN Symposium on Principles of Programming Languages (POPL 2017). pp. 175–189. ACM, New York, NY, USA (2017). https://doi.org/10.1145/3009837.3009850
23. Karr, M.: Affine relationships among variables of a program. Acta Informatica **6**(2), 133–151 (1976). https://doi.org/10.1007/bf00268497
24. Lahav, O., Vafeiadis, V., Kang, J., Hur, C.K., Dreyer, D.: Repairing sequential consistency in C/C++11. In: Proceedings of the 38th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI 2017). pp. 618–632. ACM, New York, NY, USA (2017). https://doi.org/10.1145/3062341.3062352
25. Lee, S.H., Cho, M., Podkopaev, A., Chakraborty, S., Hur, C.K., Lahav, O., Vafeiadis, V.: Promising 2.0: Global optimizations in relaxed memory concurrency. In: Proceedings of the 41st ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI 2020). pp. 362–376. ACM (2020). https://doi.org/10.1145/3385412.3386010
26. Mastroeni, I., Zanardini, D.: Data dependencies and program slicing: From syntax to abstract semantics. In: Proceedings of the 2008 ACM SIGPLAN Symposium on Partial Evaluation and Semantics-based Program Manipulation (PEPM 2008). pp. 125–134. ACM, New York, NY, USA (2008). https://doi.org/10.1145/1328408.1328428
27. McKenney, P.E., Jeffrey, A., Sezgin, A., Tye, T.: Out-of-thin-air execution is vacuous. ISO/IEC JTC1/SC22/WG21 paper P0422R0 (2016), https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2016/p0422r0.html

28. Müller-Olm, M., Seidl, H.: Analysis of modular arithmetic. In: Programming Languages and Systems (ESOP 2005). pp. 46–60. LNCS, Springer (2005). https://doi.org/10.1007/978-3-540-31987-0_5
29. Paviotti, M., Cooksey, S., Paradis, A., Wright, D., Owens, S., Batty, M.: Modular relaxed dependencies in weak memory concurrency. In: Programming Languages and Systems (ESOP 2020). Lecture Notes in Computer Science, vol. 12075, pp. 599–625. Springer (2020). https://doi.org/10.1007/978-3-030-44914-8_22
30. Pichon-Pharabod, J., Sewell, P.: A concurrency semantics for relaxed atomics that permits optimisation and avoids thin-air executions. In: Proceedings of the 43rd Annual ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL 2016). pp. 622–633. ACM, New York, NY, USA (2016). https://doi.org/10.1145/2837614.2837616
31. Podgurski, A., Clarke, L.A.: A formal model of program dependences and its implications for software testing, debugging, and maintenance. IEEE Transactions on Software Engineering **16**(9), 965–979 (1990). https://doi.org/10.1109/32.58784
32. Podkopaev, A., Lahav, O., Vafeiadis, V.: Bridging the gap between programming languages and hardware weak memory models. Proceedings of the ACM on Programming Languages **3**(POPL) (2019). https://doi.org/10.1145/3290382
33. Reps, T., Horwitz, S., Sagiv, M.: Precise interprocedural dataflow analysis via graph reachability. In: 22nd ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL 1995). pp. 49–61. ACM (1995). https://doi.org/10.1145/199448.199462
34. Richards, J., Wright, D., Cooksey, S., Batty, M.: Symbolic MRD: Dynamic memory, undefined behaviour, and extrinsic choice. Proc. ACM Program. Lang. **9**(OOPSLA1), 1858–1882 (2025). https://doi.org/10.1145/3721089
35. Schwartz, E.J., Avgerinos, T., Brumley, D.: All you ever wanted to know about dynamic taint analysis and forward symbolic execution (but might have been afraid to ask). In: 2010 IEEE Symposium on Security and Privacy. pp. 317–331. IEEE (2010). https://doi.org/10.1109/sp.2010.26
36. Stern, A., McKenney, P.E., Wong, M., Michael, M., Brito, G., Cooksey, S., Batty, M.: P3692R4: How to avoid OOTA without really trying. ISO/IEC JTC1/SC22/WG21 paper (2026), https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2026/p3692r4.pdf, 26 March 2026
37. The Clang Team: DataFlowSanitizer Design Document. LLVM Project, https://clang.llvm.org/docs/DataFlowSanitizerDesign.html, accessed 10 October 2026
38. Urban, C., Müller, P.: An abstract interpretation framework for input data usage. In: Programming Languages and Systems (ESOP 2018). LNCS, vol. 10801, pp. 683–710. Springer (2018). https://doi.org/10.1007/978-3-319-89884-1_24
39. Vafeiadis, V., Balabonski, T., Chakraborty, S., Morisset, R., Zappa Nardelli, F.: Common compiler optimisations are invalid in the C11 memory model and what we can do about it. In: Proceedings of the 42nd Annual ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL 2015). pp. 209–220. ACM, New York, NY, USA (2015). https://doi.org/10.1145/2676726.2676995
40. Wright, D., Dalvandi, S., Batty, M., Dongol, B.: Mechanised operational reasoning for C11 programs with relaxed dependencies. Formal Aspects of Computing **35**(2), 1–27 (2023). https://doi.org/10.1145/3580285
41. Yadegari, B., Debray, S.: Bit-level taint analysis. In: SCAM 2014. pp. 255–264. IEEE (2014). https://doi.org/10.1109/SCAM.2014.43
42. Yasuoka, H., Terauchi, T.: Quantitative information flow – verification hardness and possibilities. In: 23rd IEEE Computer Security Foundations Symposium (CSF 2010). pp. 15–27. IEEE (2010). https://doi.org/10.1109/csf.2010.9

Appendix

The appendix is supplementary material for reviewers and is not part of the proceedings version. It holds the proofs (also checked by Isabelle/HOL), the application's definitions and hand lemmas, and a record of what was executed; every result and its status is stated in the main text.

A Proofs

Proof of Thm. 1. The shared second instruction contributes the same summary. For the first, $x{+}1$ and $x{+}x$ are both non-constant in x ($0{+}0 = 0$, $1{+}1 = 2$) and mention no other register, so both summaries are $(t, \{x\})$. Yet P_+ gives $y = (x{+}1) - x = 1$ and $P_\times$ gives $y = x$. □

Proof of Thm. 2. Fix $\beta \in \mathcal{A}_{\text{read}}$ and distinct t, y. (The Isabelle statement carries the hypothesis $t \neq y$ to build its witnesses; the conclusion does not mention t, y.) Each witness is $P_E^g = [(t, e_E), (y, g)]$, so $\alpha_\beta(P_E^g) = [(t, S_E), (y, S_g)]$ with $S_E = \{r \mid \beta(e_E, r)\}$. Take $e_A = 0$, $e_B = x$, $e_C = x \,\&\, 1$, $g_1 = t$, $g_2 = t \,\&\, 2$. Since $S_E \subseteq \mathit{regs}(e_E)$, $S_A = \emptyset$ and $S_B, S_C \in \{\emptyset, \{x\}\}$. *Case* $S_B = \emptyset$*:* $P_A^{g_1}$ computes 0 and $P_B^{g_1}$ computes x; the summaries are equal and $\mathit{prog_dep}(\cdot, x, y)$ differs. *Case* $S_C = \emptyset$*:* likewise with $x \,\&\, 1$. *Case* $S_B = S_C = \{x\}$*:* $P_B^{g_2}$ computes $x \,\&\, 2$, which varies, and $P_C^{g_2}$ computes 0. □

Thm. 3, details. (a) Right to left, equal denotations give equal runs. Left to right, with s_0 separating e_1, e_2 and x, t fresh, the context $[(t, \cdot), (x, (t \neq e_1) * (x{+}1))]$ yields 0 for e_1 and $v{+}1$ for e_2 from $s_0[x{\mapsto}v]$. (b) If $c(e_1) = c(e_2)$ with distinct denotations, the context of (a) gives equal summaries and different dependence. Conversely, a code that refines denotations is exact, because f can replay the program through a representative of each code. (c) Distinct (a, b) are separated at $x = 0$ or $x = 1$; injectivity on the 2^{64} pairs gives the count (`affine_capacity_card`). (d) Register names are strings and expressions are countable. *Full one-register count (hand argument, not mechanised):* $E_f = \sum_n (x{=}n) * f(n)$ reads only x and denotes any $f : \{0,1\}^{32} \to \{0,1\}^{32}$. So by (b) an exact code needs $2^{32 \cdot 2^{32}}$ values on such instructions.

NP-completeness of *prog_dep* (hand argument, not mechanised). Membership: guess s and two values of a and evaluate. Hardness: write a Boolean formula φ with &, | and $\oplus 1$ over the bits $z_i \,\&\, 1$; then $[(y, x{*}\varphi)]$ has $\mathit{prog_dep}(x, y)$ iff φ is satisfiable.

Thm. 5, details. (a) The induction hypothesis is the generalisation "if s_1, s_2 agree on $\mathit{vdep}(P, y)$ then $\mathit{run}\, P\, s_1\, y = \mathit{run}\, P\, s_2\, y$". The step uses the fact that $[\![e]\!]_{s_1} = [\![e]\!]_{s_2}$ whenever s_1, s_2 agree on $\{r \mid \mathit{nc}(e, r)\}$, a finite hybrid argument over $\mathit{regs}(e)$. The inclusion in sites^* is a second induction. (b) Induction shows $\mathit{vdep}(P, y) = \{x\}$ and $\mathit{run}\, P\, s\, y = h(s\, x)$ with h injective: a step that defines the current source composes h with the injective map $v \mapsto [\![e]\!]_{[r \mapsto v]}$, which depends only on r because $\mathit{regs}(e) \subseteq \{r\}$. Injectivity of h then separates $x \mapsto 0$ from $x \mapsto 1$.

Thm. 6, details. For one affine expression, $[\![e]\!]_{s[r\mapsto v]} = \mathit{coef}(e,r)\cdot v + [\![e]\!]_{s[r\mapsto 0]}$, by induction on e; hence $nc(e,r) \iff \mathit{coef}(e,r) \neq 0$. (a) is induction on P with the step (z,e) split on $r = z$: for $r \neq z$ the induction hypothesis is used once for r and once for z, and the two contributions are $w(Q,b,r)\cdot v$ and $w(Q,b,z)\cdot \mathit{coef}(e,r)\cdot v$. Taking $v \in \{0,1\}$ gives the dependence characterisation, and soundness of *vdep* gives the inclusion. (b) w depends on a step only through its target and coefficient vector; the separating context has weight $\mathit{coef}(e_i,x) - \mathit{coef}(e_1,x)$. (c) For the converse direction f evaluates w on a representative of each code value. The map $a \mapsto c(a\cdot x)$ is injective for every exact c, and *coef* has exactly 2^{32} values on $\{a\cdot x + b\}$ (`aff_capacity_classes`).

Thm. 4, details. If $\mathit{run}\, P_1\, s_0\, z \neq \mathit{run}\, P_2\, s_0\, z$, let *pre* set every register of P_1, P_2 and z to its value in s_0, and $\mathit{post} = [(x, (z \neq k_1) * (x{+}1))]$ with $k_1 = \mathit{run}\, P_1\, s_0\, z$ and x fresh: the final x is constantly 0 after P_1 and equals $x{+}1$ after P_2. A block's final z depends only on the registers it mentions and z; the pinning prefix assigns literals to these, so after it both blocks produce the constants they produce from s_0, and the fresh x is untouched. For (c), $[(t,e)]$ has $\mathit{prog_dep}(a,b)$ iff $b = t \wedge nc(e,a)$ or $b \neq t \wedge a = b$, and x and $x \,\&\, 1$ are non-constant exactly in x; at $x = 2$ they differ.

Remarks. By Thm. 5(e), *vdep* can be inexact for (x,y) only if some step reads two marked registers (cancellation across registers), a non-constant one (masking, range restriction) or one that a side input can annihilate; sites^* also loses precision at overwrites (d). Part (e) is proved by a forward invariant: each register's class bounds how its value varies with x when all other inputs are fixed (`taint_sound`). Expanded composed labels can grow exponentially on reused temporaries, so they should be shared definitions.

Thm. 7, details. Input symbols are injective and disjoint: I_r per initial register and L_i per load. Symbolic execution agrees with concrete execution (`label_store_val`), and every concrete input is realised by some valuation, which gives (a). For (d), symbolic execution from the environment that binds each register to its fixed value agrees with concrete execution from s (`label_at_store_val`); the existential form is immediate from the definitions. The embedding maps *run* to concrete execution, which gives (b). (c) follows from (b), Thm. 2 and the fact that the embedding's event summary is determined by $\alpha_\beta(P)$. Positive controls are proved too: a stored load is a dependency, a load overwritten by a constant is not, and after a reload only the second load counts.

Per-site correspondence with a transcription of PwT's R4b. With $R4b_\rho \iff [\![e]\!]_\rho \neq [\![e]\!]_{\rho[r\mapsto 0]}$, we have mechanised that non-constancy holds iff $\exists\rho.\, R4b_\rho$, and that $R4b_\rho$ fails whenever $\rho(r) = 0$ (theory `PwT_R4b`). That $R4b_\rho$ renders PwT's R4b is a modelling assumption. PwT's actual check also uses local values, which makes it finer.

B The Application: Definitions and Pen-and-Paper Lemmas

Text relocated from §5 and §6 (space). *Related analyses.* Hunt and Sands [18] prove optimality within a flow-sensitive type family and show semantic incompleteness by example. Abstract-interpretation dependency analyses track read sets [4] or gain precision from value domains [5,38]. *Rings.* We place each of 1,828 enumerated single-register expressions e at every store of an n-thread LB ring, $n \in \{2, 3\}$, and ask for all reads to observe 1. The generator keeps exactly the expressions with $e(0) = 1$. Verdicts are identical at $n = 2$ and $n = 3$ for every expression, so we count expressions. 1,184 have $e(1) \neq 1$ and are (W). Of the 644 value-consistent expressions, 534 are constant in r, and SDEP admits their rings. The other 110 are non-constant with $e(1) = 1$, and SDEP forbids their rings (D). Folding (A) also forbids 385 of the 534, not seeing their constancy; treating every syntactic dependency as semantic forbids all 534. As $e(0) = 1$, the 110 forbidden rings are not thin air, and by hand derivation Promising admits them (below); per-thread non-constancy is stricter. *Corpus.* From tests distributed with herd7 and Dat3M, 2,396 files are *eligible* for the encoder (no branches, RMWs, sub-word accesses or non-conjunctive outcomes); without the 16 AArch64 virtual-memory files, 2,380 raw files are *analysed* (2,340 byte-distinct). SDEP allows 2,345 and forbids 33, and 2 files produce no memory events. Modes A and B give the same verdict on every file, and mode C agrees on all 2,250 files it supports; five of the 33 contain address edges, which mode C does not decide. Of the 33, 31 come from the coherence base (B) and 2 from same-location write ordering (W); none is forbidden by the dependency axiom alone. Only treating every syntactic dependency as semantic changes verdicts: in 6 files (App. D) it turns ALLOWED into FORBIDDEN, as hardware that orders syntactic dependencies does.

Basic properties. On branch-free candidates: (a) If each thread of a two-thread LB candidate has one read and one write and both stores are constant in their reads, then $\mathsf{sdep} = \emptyset$, so $\mathsf{sdep} \cup \mathsf{rf} = \mathsf{rf} \subseteq W \times R$ is acyclic. (b) $\mathsf{sdep} \subseteq \mathsf{po}$, so $\mathsf{acyclic}(\mathsf{po} \cup \mathsf{rf})$ implies $\mathsf{acyclic}(\mathsf{sdep} \cup \mathsf{rf})$. The converse fails on T_+-LB.

The 110 forbidden ring expressions (hand derivation; no checker run). Because $e(0) = 1$, value 1 in these rings is justified from the initial state, so they are not thin-air outcomes. By hand derivation from the definitions, Promising and Promising 2.0 admit the outcome (a thread promises to write 1 and certifies the promise by reading the initial 0), a compiler that knows every location holds 0 or 1 may fold the store to 1 and reorder it, and RC11 and dependency-ordering hardware forbid it. The criterion's verdict is a consequence of deciding dependency per thread over all 32-bit values, not an implementation error, and not an instance of preventing thin air.

LB-mem-cancel under Promising (hand derivation; no tool run). Thread 0 promises $x{:=}1$. Certification must succeed from every future memory (Promising) or from the capped memory (Promising 2.0). In either, thread 0 alone can read $y{=}0$ from its view, write $z{:=}1$ at a fresh timestamp (in 2.0, above the cap), reload its own write ($u{=}1$) and write $1-0 = 1$ to x, fulfilling the

promise; no other thread's write is needed. Thread 1 reads the promised x=1 and writes y:=1; thread 0 reads y=1, writes z:=2, reloads 2 (its view of z already includes that write) and writes $2-1=1$, fulfilling the promise. So the outcome r_0=r_1=1 is admitted. The tool-emitted facts are separate: mode C forbids, the same encoding with $\mathsf{sdep}=\emptyset$ admits, and the single well-formed candidate has the cycle $\mathtt{ld}\,y \rightarrow_{\mathsf{sdep}} \mathtt{st}\,x \rightarrow_{\mathsf{rf}} \mathtt{ld}\,x \rightarrow_{\mathsf{sdep}} \mathtt{st}\,y \rightarrow_{\mathsf{rf}} \mathtt{ld}\,y$. Only thread 0 writes z, and reading the initial 0 after that write is a coherence cycle (fr against $\mathsf{po}_{\mathrm{loc}}$), so $u = r{+}1$ for every 32-bit r.

LB-mem-cancel and store forwarding (constructed tests; tool runs). Read with C11 `memory_order_relaxed` accesses (unsigned values), three separate steps lead to a program that exhibits the outcome. *Bridge.* Vafeiadis et al. prove their transformation results for *monotone* models [39, Def. 3], which exclude the 2011 model (ConsRFna, SCorig, RSorig, STorig). But in a program whose accesses are all relaxed and whose initial writes happen before every access, ConsRFna (which constrains only rf edges with a non-atomic end) always holds and the SC axioms are vacuous, so its consistent executions under the 2011 model are those under the monotone (Naive, SCnew, RSorig, STorig); all three programs below are of this kind. Two obligations make this an equality of observable behaviours. (i) It holds for every opsem of the program's opsemset, prefixes included: a prefix keeps the initialising writes and is still all-relaxed, so the argument applies to it unchanged. (ii) The initialising writes precede the parallel composition (asw), hence happen before every access, so they race with nothing; *Racy* is defined without reference to the model, so undefined behaviour coincides too. Hence a transformation from one program of this kind to another (step (1) adds no non-atomic access) is sound for the 2011 model iff it is for the monotone one. (1) Replace the reload $u \leftarrow z$, which immediately follows the store to z, by $u := r{+}1$: sound by [39, Thm. 10] (adjacent read after write, any read mode under STorig), hence sound here for the 2011 model; Thm. 10 equally covers the variant with Arf, which forbids $\mathsf{hb} \cup \mathsf{rf}$ cycles. (2) Fold $(r{+}1){-}r$ to 1: register arithmetic, no memory event changes. (3) Move the store x:=1 above the store to z and the load of y: valid for Naive ([39, Table 1, Thm. 6] with monotonicity), invalid for Arf and Arfna, which forbid moving a write above an earlier read. On the original, the result of (1) and the result of (3), modes A/B/C give F/F/F, F/F/**A** and A/A/A, and $\mathsf{sdep}=\emptyset$ admits all three; RC11 forbids the first two. After (1) the store's label is $(L_r{+}1){-}L_r$, constant, as in T_+. So, read as C11 relaxed atomics, the criterion is not invariant under an elimination sound for this program; the outcome itself is allowed by the 2011 model throughout, which does not exclude load buffering. For the test as written in LKMM, step (1) is not a compiler step (`READ_ONCE` and `WRITE_ONCE` are volatile) and LKMM orders the chain itself: $\mathtt{to\text{-}r} = (\mathit{addr}; [R]) \cup (\mathit{dep}; [\mathit{Marked}]; \mathit{rfi})$ is part of its *ppo*, as $(\mathit{addr} \cup \mathit{data}); \mathit{rfi}$ is part of Arm's *dob*. Compiled as C11 relaxed atomics by clang 20.1.2 at `-O0` to `-O3` for four targets, the program keeps both relaxed loads; with z non-atomic, clang at `-O1` to `-O3` forwards the stored value and stores the constant 1 (App. D). No Promising checker was executed. PwT does not validate store forwarding [21, p. 54:28]; MRD has a rule for it [29, §4.1].

Batty et al.'s witnesses. Batty et al. [7] argue that no criterion over the unchanged C11 candidate handles both of their witnesses; ours, like MRD's and PwT's, carries program-derived labels, and their witnesses differ in branch structure, outside $\mathcal{S}$.

Encoding adequacy. Fix a finite candidate with all events active, its rf, co and the supplied dependency relation C. The consistency formula is satisfiable iff the candidate satisfies per-location coherence, RMW atomicity and $\mathsf{acyclic}(C \cup \mathsf{rf})$. Acyclicity is encoded as an integer ranking with $u \prec v$ on every edge, which exists iff the relation is acyclic. Each conjunct uses fresh ranking variables. A legacy conjunct over a separate integer family uses only non-strict inequalities and is satisfied by the all-zero assignment.

Lemma (well-formedness adequacy). For a finite, all-active candidate with rf and co fixed, the well-formedness formula is satisfiable iff:

- each read has exactly one same-location rf source with matching value;
- co is a strict total order per location, with the initial write first;
- the outcome and final-memory atoms hold;
- per location, $\mathsf{po} \cup \mathsf{co}$ is acyclic.

Proof idea: every ordering constraint relates two events of one location, and strict orders on disjoint per-location event sets extend to one global order. The last condition is why the (W) class of §5 contains same-location write-ordering failures, such as `CoWW`. The lemma depends on keeping cross-location program-order edges out of this order; an encoder that includes them forbids legal candidates (the artifact documents such a case).

C Control Dependence on a Delimited Fragment

For one controlling read, one two-way branch, and arms that each write once to the same fixed location, we encode the branch's write effect as the expression $(g \neq 0) * e_t + (1 - (g \neq 0)) * e_f$. With semantic control dependence defined as value dependence of this merged write, non-constancy of the expression decides it (`P3_exact_control_on_F0`). That follows from the encoding by definition. Three mechanised witnesses show that the one-word effect abstraction fails beyond this fragment. The front end does not build this encoding, so no experiment exercises it.

D Evidence: What Was Executed

Independent reference. Agreement through a shared solver is not evidence of correctness, so we wrote a separate reference in Python. It shares no code with the prototype, though it uses the same solver for one direction: it has its own litmus reader, expression parser, symbolic executor and bit-vector evaluator. A "varies" verdict needs a concrete witness, two load values that make the store differ, re-evaluated without a solver. A "constant" verdict is an unsatisfiability

proof from Z3 over a term the reference builds itself, so in this direction the solver is shared. It is corroborated by exhaustive evaluation at 8 bits and 20,000 random 32-bit trials. The reference decides each pair at 32 and at 64 bits.

Population and naming. Of the 80 injective-pipeline pairs, 48 rewrite one register and fall under Thm. 5(b); the other 32 read two registers and are real by construction. Three fake check pairs carry no syntactic edge (an overwrite or reload kills the load). The artifact calls the SDEP model WEAKEST, a name we avoid because Chakraborty and Vafeiadis use it for a different model [10].

Solver-free validator. A third checker, with its own parser and 32-bit evaluator and no solver, checks every composed-expression verdict that mode C records (684 edges: 38 diagnostic, 6 regression, 314 population, 326 corpus); it does not check mode L. Four kinds of evidence result. *Concrete witness:* all 202 "varies" verdicts, with a 32-bit witness the validator finds itself (the reference finds its own). *Exact normalisation:* 462 of the 482 "constant" verdicts are *proved* by algebraic rewriting (ring normal form, $a \oplus a = 0$, constant masks) that removes the load symbol: all 322 corpus, 123 population, 15 diagnostic and 2 regression fakes. *Finite testing only:* the other 20, population masking expressions such as $((r * 8)\,\&\,2)$, pass exhaustive enumeration at 8 bits and a deterministic 32-bit search; a variation found at 8 bits would lift to 32 bits for these comparison-free operators, but no variation found is not a proof. *SMT:* every "constant" verdict, the 20 included, is a Z3 unsatisfiability proof in the reference, the solver the prototype also uses. The validator and this decision procedure both start from the expression mode C recorded, so they check the verdict, not the front end. A separate front end rebuilds the expression from source only where the reference parses the input (C syntax: 6 regression, 38 diagnostic, 314 population and 1 corpus edge), and Z3 then proves the two expressions equal.

All checks below were run on Windows 11 with the Java Development Kit (JDK) 21, Z3 4.13 (via JavaSMT 5.0.1), Python 3.11 with the z3-solver package (Z3 5.0.0) for the reference, and Isabelle2024. They were run from a fresh extraction of the artifact archive, whose hash manifest verified.

1. *Isabelle.* All three sessions build. A check theory verifies, for all 73 cited facts, that no oracle is used, and that none of the 14 theories involved has `WEV_SMT_Defs` as an ancestor. The fourth session, `WEV_SMT_TL` (Thm. 5(e)), is in the fourth archive with byte-identical copies of the theories it builds on; all four sessions were rebuilt from scratch with exit status 0. Its check theories confirm that its 31 cited facts use no oracle and that `WEV_SMT_Defs` is not an ancestor of `PwT_TaintLinear`; a control confirms that the same test flags a theory that does import it.
2. *Composed-label mode.* We ran it on the diagnostic inputs, the 256-file population and the 2,396 eligible corpus files, recording a per-file support and exclusion manifest. We ran the independent reference and the expression-equality check on the same files, and mode L on the diagnostic inputs and the population. The 21 diagnostic inputs are the six LB programs of Table 3, `LB-direct-cancel` (`LBfd` in C syntax) and 14 unit cases: overwrites, copy chains, reloads, overflow, masks, signed comparisons and three rejected constructs. On them the reference

agrees on all 38 pairs and rejects the same three files. One unit case, $s := (r * 65536) * 65536$, is constant at 32 bits and varies at 64. Expression equality: for each edge, the prototype's printed expression is parsed by the reference, its load symbols are renamed through the event table, Z3 proves its difference from the reference's expression unsatisfiable at 32 bits, and 2,000 random evaluations agree (6 regression, 38 diagnostic, 314 population and 1 corpus edges). Three regression inputs, $r_0 :=$ `load`; $s := r_0 * k$; `store` s with initial $k = 0$, $k = 1$ and k undeclared, confirm that both tools fix initial registers as $esdep_s$ does: constant, varying, constant. Mode C rejects a file, with its reason, if it contains an address or control edge, an RMW, a page-table token or lies in the virtual-memory directory, an expression outside the oracle's grammar, or a construct the encoder cannot represent. Each load symbol in a composed expression must be a syntactic data predecessor of the store; a dependency omitted from both surfaces only where the reference rebuilds the expression from source. Neither check found a case. On the corpus, our reference parses 611 of the 633 C files (not initialised declarations or plain pointer stores); they hold one data pair, on which the tools agree. The other 325 corpus edges, in assembly tests, are checked only from their recorded expressions. In the population, the generator's validator re-derived all 317 template-stated labels with its own Z3 encoding (real ones confirmed by execution); all agree. The 66 single-instruction fakes have 41 distinct shapes, and the 72 chain-form fakes have 52.

3. *Memory hop.* A separate input set, kept out of all corpus, population and ring counts, holds `LB-mem-cancel` and T_+-LB. The driver runs modes A/B/C, the reference and the six models on it, and a witness driver (ours, compiled against the pinned classes) enumerates every rf assignment, checks each for well-formedness, for SDEP with $\mathsf{sdep} = \emptyset$ and with mode C's sdep, prints Z3's model, and searches $\mathsf{sdep} \cup \mathsf{rf}$ for a cycle. For `LB-mem-cancel`, one of 8 assignments is well-formed (values r=1, z=2, u=2, x=1); it is admitted with $\mathsf{sdep} = \emptyset$ and forbidden under C. With the outcome replaced by "the reload of z reads 0", the 4 such candidates are well-formed but violate coherence. The two encodings share encoder and formula and differ only in the sdep set passed to $\mathsf{acyclic}(\mathsf{sdep} \cup \mathsf{rf})$, which is the only place the corrected build uses it. The checker fails if either side of the contrast changes.
4. *Supplement (outside every total above).* A third archive holds the upstream search and the forwarding runs, with a checker. The search protocol was written before either source was fetched: McKenney's LKMM litmus collection at commit `2f1be9ed` (5,571 files; 1,066 byte- or whitespace-identical to our corpus are excluded) and the InstCombine tests of LLVM `llvmorg-22.1.0`. A candidate needs a load reaching a store through two or more assignments (litmus: in a file mode C supports; LLVM: through pure arithmetic, with the expected output dropping the dependence by a fold valid under wrap-around, in the paper's grammar). Litmus: mode C supports 1,898 of 4,505 files (rejections: RCU, control, locks, plain accesses and others); A, B and C agree on every edge and verdict; 2 files have a data edge, both copies; none is eligible. A parser-independent text scan finds 33 files with a load-to-store

data path and one through two assignments, `C-JO-OOTA-4` (rejected by the parser), whose label $((L_1{=}0){+}1)\,\&\,L_2$ Z3 shows non-constant in each load at 32 and 64 bits. LLVM: 1,767 files, 344 functions with a load and a store, 59 such paths in 18 functions; one expected output drops the dependence, and it does so by folding loads from an uninitialised `alloca` (undef), which under wrap-around varies (Z3). Nothing was eligible, so no harness was built and no B/C comparison was possible; the counts describe these two pinned sources only. The checker re-runs the forwarding tests against their expected verdicts; the LLVM script (InstCombine alone, and NewGVN then InstCombine) and its output are included but not run by it. On 32-bit IR, LLVM 22.1's InstCombine rewrites the context of Thm. 3(a) to 0 after $2x$ and to $u{+}1$ after $2x{+}1$ but not after $2x{+}2$, which NewGVN then InstCombine also rewrites to $u{+}1$: IR simplification, not a dependence verdict.

5. *Argument-to-return comparison (supplement; functions chosen after the search, in an internal review; outside the search and every total).* The four functions (`add.ll` `@test26` and `@test35`, lines 769–776 and 919–926; `xor.ll` `@test19` and `@test4`, lines 174–181 and 43–50; `llvmorg-22.1.0`) are stored verbatim with their CHECK lines; all are i32 with no `nsw`/`nuw` flag and no undef or poison, so add, sub and xor wrap modulo 2^{32} as in the prototype. The `apint-sub.ll` `@test19` case (i39) is kept out of the 32-bit results. Our constructed wrappers load each argument and store the returned value. Mode B keeps the disputed edge and mode C removes it in 4 of 4; the other argument's edge (`@test26`: A; `xor` `@test19`: B) is kept by both. The reference agrees on every pair. A separate script parses the IR text itself and decides each argument: a non-constant result by a re-evaluated witness, a constant one by Z3 over its own term, with exhaustive evaluation over all 2^{32} values for the one-argument functions and at 8 bits for the others. We ran InstCombine itself (the pass of LLVM 22.1.0, in-process through llvmlite 0.50.0, not the `opt` binary) on each function alone and on its whole test file: it returns the other argument or -1 and no longer references the disputed argument, matching the CHECK lines. Each wrapper is admitted in every mode. A regression test requires a mutant $((A{+}B){-}A)$ to flip both edges and eight corrupted copies of the outputs to be rejected. The search counted 35,946 InstCombine functions and a separately written script 35,953; the gap is the counting rule (the search's pattern misses 8 definitions: six follow a header with text after its brace, two end the file with an indented brace; the script misses one after a one-line function), none of the 8 has a load or a store, and no rate uses either total; both are 35,954 definition lines minus the misses.
6. *Compiled code (recovery supplement; constructed; outside every total).* One generator emits each of 17 patterns (14 cancelling, 3 dependent) as an LKMM litmus file and as C11 with `atomic_uint`, relaxed loads and stores and unsigned arithmetic; Z3 confirms each cancelling store constant and each control not. Modes A/B/C (pinned build) and the reference were run on the litmus files. Clang 20.1.2 (bundled with zig 0.15.2) compiled the C at `-O0` to `-O3` for four targets. Zig maps `-O1` to `-O3` all to clang's `-O2`, so these three columns were one compile; replaying zig's clang command at genuine `-O1`, `-O2` and `-O3`

(fidelity-checked against zig's own output) gives identical IR for all 17 programs on all four targets, and the same verdicts; a script follows SSA def-use from each `store atomic` back to `load atomic` in the IR, and register dataflow from `str` to `ldr` in the AArch64 assembly. At `-O1` to `-O3` every target agrees; `-O0` keeps values in stack slots, which the script does not follow, so it is not counted. The Arm verdict uses only that, with no barrier, acquire or release, the load-buffering cycle is in Arm's ordering iff both threads' stores depend on their loads. A second, separately written script parses the AArch64 assembly at `-O1` to `-O3`, builds $ob = rfe \cup dob \cup bob$ on the load-buffering candidate, and agrees on all 17 programs and on 7 variants (renamed registers and locations, other constants, swapped operands, 3-thread rings); the compiled code contains no barrier, acquire, release, exclusive or branch instruction. A supplement theory (`WEV_SMT_Rec`; a corollary, no oracles) proves that program-level and event-level dependence are invariant under any rewrite preserving the stored values, such as clang's fold of $(r+1)-r$, whereas the version-aware closure and $sites^+$ relate r to the store before that fold and not after it. On `LB-mem-cancel` clang keeps both relaxed loads at every level and target; with z non-atomic, at `-O1` to `-O3`, it forwards the stored value and stores the constant 1.

7. *Code-base screen (fourth archive; outside every total above).* The protocol was frozen before any screening; nine amendments are recorded. A marked load is an LLVM `load atomic` or `load volatile`; a sink is the value of an atomic or volatile store or RMW (D, the paper's fragment), the value of a plain store (P, exploratory), an address (A) or a branch or select condition (K). Each module was compiled by clang 20.1.2 at `-O0` without `optnone` for x86-64 and screened in two views: scalar replacement only (SL), and after force-inlining every function of at most 400 instructions except `main` (SLI). On the backward slice from a sink to the load, the screen decides the composed question by self-composition in QF_BV at the IR's widths, with calls, other loads, floating point and inline assembly as shared uninterpreted functions, and the closure's question by the same oracle per instruction; a 5 s timeout gives UNKNOWN and a slice over 300 gives LARGE. So "fake" is sound by construction and "real" conservative: clang also folds three sites the screen classes real. Of 1,743 modules in the eight sources 1,242 compiled, and five SLI jobs ran out of memory or time (mimalloc 2, radix-tree harness 3). SLI totals: D 501 sites (497 real, 3 fake at one instruction, 0 composed cancellations, 1 LARGE); P 416 (0 composed, 21 LARGE); A 930 (2 composed, 107 LARGE); K 5,559 (50 composed, 110 UNKNOWN or LARGE). The SL view has no composed cancellation. Not screened: the nidhugg tests and a second vma configuration (memory), the BPF ring buffer (build failure), DPDK, and the GCC and LLVM test suites; results of a superseded tool version on some of them are not used. An unmechanised multi-width LLVM analogue of the classification of Thm. 5(e), run as a census on the screened pairs, passes a check fixed before any corpus result on all 124,756 joined pairs (86,856 without duplicate builds): no pair it marks injective or non-constant is classed fake, and every composed cancellation is marked unknown. None of the 282 pairs behind the 52 composed cancellations has a correlation origin; by the recorded triage they come from

maxima, remainders, masks and bit tests, or a dead path. Of the 52 composed cancellations, 44 lie in single-threaded unit tests or guard only stack objects, one is a dead path (`node = NULL`), and seven remain: five libvsync assertions with a `seq_cst` source, for which the recorded argument (AArch64, x86-TSO, RC11) is that the dependency adds no ordering; one liburcu branch dominated by real branches on the same load; and one in a test harness's stale copy of a kernel function, whose kernel version has since changed its parameter type. The rule that an acquire-or-stronger source makes a dependency redundant was written before triage but after the protocol was frozen. Under Arm's architectural notion of control dependence two further Concurrency Kit sites remain, both with a `seq_cst` RMW source. Replaying zig's clang command at genuine levels (fidelity-checked), clang 20.1.2 removes every cancelled instance at the six library sites at `-O1`, `-O2`, `-O3` and `-Os` for x86-64 and AArch64; one instance of the same source line on a pointer the screen classes real survives. A source-level screen of Linux 7.3 found 573 load-to-marked-store pairs, all real, but 6,707 of the 8,386 relevant files had parse errors. Transcribed examples from the kernel's documentation (`rcu_dereference.rst`) include one composed cancellation, $x - (\texttt{uintptr_t})x$, which the documentation itself warns against. A verifier re-checks 43 numerical claims of the screen's reports against its outputs; all pass.

8. *Models.* We executed the six encoded models (SC, TSO, PSO, Coh, which the artifact calls RA, SDEP and RC11) and the well-formedness-only query on Table 3. The SDEP verdicts on the corpus equal those the base artifact lists for all 2,394 files that encode.
9. *Rings.* All 3,656 rings were executed with the prototype and equal the rows the base artifact lists. The partition into 1,184/534/110 expressions was recomputed by the reference.
10. *Corpus counts.* The 16 excluded files are selected by directory. A script recounts them, scans their contents for page-table tokens (found in 11), and recomputes the totals from the fresh verdicts. Another reports raw, byte-distinct and normalised counts and lists every file whose verdict changes between two configurations. The 6 files whose verdict changes when every syntactic dependency is semantic are 5 supported by mode C (Power and AArch64 `LB+datas`, two AArch64 variants and `DETOUR0236`) and `LB+addrs`, whose fake $r \oplus r$ dependency is an address.
11. *Not re-executed.* The population's inputs and the generator's labels are archived. The driver recomputes their structure (shapes, assignment counts, templates) from the input text and checks the archived labels against the fresh reference verdicts. The ring verdicts (identical at $n = 2, 3$; the 1,184 value-inconsistent; SDEP admits the 534; folding forbids 385; SDEP forbids the 110) are read from the archived ring audit, which the full re-execution reproduces row for row; the reference's classification must agree with it on every expression. The Promising comparison, the multi-thread criterion and the encoder's adequacy are hand derivations, and literature claims are not checked by the artifact.
12. *Checks of the checks.* The driver also corrupts copies of its outputs (70 cases: disagreements, missing, duplicate, malformed, truncated or coherently rela-

belled records, unmatched edges, expressions that no longer fit their verdicts, stale summaries, a wrong base archive, missing evidence, missing or malformed proof and ring logs, either side of the memory-hop contrast flipped) and requires every corruption to fail, and perturbs composed expressions to confirm that the equality check detects them.

Comparisons with herd7 and Dat3M are in the base artifact but are not used in this paper.

Table 4. Mechanisation in Isabelle2024: session `WEV_SMT_PwT`; for Thms. 4, 5(a)–(d), 6, 7(d) and `PwT_Remarks`, session `WEV_SMT_Rev` built on it; for Thm. 5(e), session `WEV_SMT_TL` built on that. For these and the other facts cited (73 facts over 14 theories, and 31 in `WEV_SMT_TL`) we checked mechanically that no oracle is used. The older development's theory `WEV_SMT_Defs` axiomatises an SMT oracle's soundness, its completeness on branch-free pairs, and that control dependencies are semantic; we checked mechanically that it is not an ancestor of any theory listed.

Result	Isabelle theorem(s)	Theory
Thm. 1	`boolean_abstraction_too_coarse`, `no_function_of_alpha_computes_prog_dep`	`PwT_Granularity`
Thm. 2	`A_read_insufficient`; as a corollary of Thm. 3: `A_read_insufficient_via_capacity`	`PwT_ReadRestricted`, `PwT_Capacity`
Thm. 3	`ctx_equiv_iff_sem_equiv`, `exact_code_iff_refines_sem`, `affine_capacity_inj`, `affine_capacity_card`, `unrestricted_beta_exact`	`PwT_Capacity`, `PwT_Remarks`
Thm. 4	`block_equiv_iff`, `exact_block_refines_run`, `run_exact_block`, `dep_relation_not_exact`	`PwT_Block`
Thm. 5	`vdep_sound`, `vdep_le_sites`, `vdep_exact_sri`, `sri_needed`, `sites_not_exact_sri`; Cousot: `crel_iff_vdep`, `overwrite_cousot_kills`; masking: `sitewise_not_complete_single_read`, `vdep_masking`; SRI not necessary: `vdep_exact_single`, `sri_not_necessary`; (e): `tl_exact`, `tl_frag_exact`, `taint_sound`, `sri_no_tu`, `sri_exact_via_tl`, `tl_needs_linear`, `tl_needs_full`, `tl_needs_uniform`, `inj_in_add_side`	`PwT_Versioned`, `PwT_Program`, `PwT_Correspondence`, `PwT_Remarks`; (e): `PwT_TaintLinear`
Thm. 6	`wdep_shift`, `wdep_exact`, `wdep_le_vdep`, `wdep_summary`, `affine_ctx_equiv`, `affine_ctx_sep`, `aff_exact_iff`, `aff_capacity`, `aff_capacity_classes`, `wdep_witnesses`	`PwT_Affine`, `PwT_Remarks`
Thm. 7	`esdep_iff_label_non_constant`, `esdep_embed_iff_prog_dep`, `event_summary_insufficient`, `event_witnesses`; (d): `esdep_at_iff_label_at_non_constant`, `esdep_iff_ex_esdep_at`, `kprog_regression`	`PwT_EventBridge`, `PwT_Correspondence`